

Round 2 - NLP Technical Report

Social Engine: Semantic Comprehension Layer Reconstruction

Team: Event Horizon | Data Vortex A'26 | 19 September 2026

0.734 Test Accuracy Baseline: 0.348	0.733 Test Macro-F1 Baseline: 0.172	0.734 Test W-F1	0.601 Cohen's Kappa Moderate agreement	0.595 Multi-seed F1 std=0.010
---	---	--------------------	--	-------------------------------------

[PRE-COMPUTED BENCHMARK - GPU TRAINED ON GOOGLE COLAB] Model: BERTweet (vinai/bertweet-base) | Fine-tuned transformer | Epochs: up to 5 (early stopping, patience=2) | Batch=16 | LR=2e-5 | Hardware: T4 GPU | Split: 70/10/20 stratified | Seed=42. Local verifier (no GPU needed): python benchmarks/bertweet/verify_benchmark.py. Full reproduction guide: benchmarks/bertweet/README.md.

1. Problem Definition

The Social Engine's semantic comprehension layer has failed, requiring reconstruction of its NLP module. Team Event Horizon was provided Dataset 2 (9,001 labelled social media posts) and tasked with training a model to automatically classify post sentiment. The task is a 3-class supervised classification problem: Positive / Negative / Neutral.

> Dataset Properties

- File: Labeled_Social_NLP_Training_Data.csv
- Total Rows: 7,900 (after deduplication / cleaning of 9,001 raw)
- Train/Val/Test: 5,530 / 790 / 1,580 - 70/10/20 stratified split, seed=42
- Class Balance: Negative: 30.8% | Neutral: 34.7% | Positive: 34.4%
- Primary Task: Multi-class Sentiment: Positive / Negative / Neutral
- Random Seed: 42 (all operations - split, model, CV, search)

Task selection rationale: The discrete categorical sentiment_label column maps directly to the evaluation rubric's Confusion Matrix requirement. Supervised classification is the strongest choice - topic modeling (unsupervised, no ground truth) and NER (token-level annotation) would produce a synthetic or misleading confusion matrix.

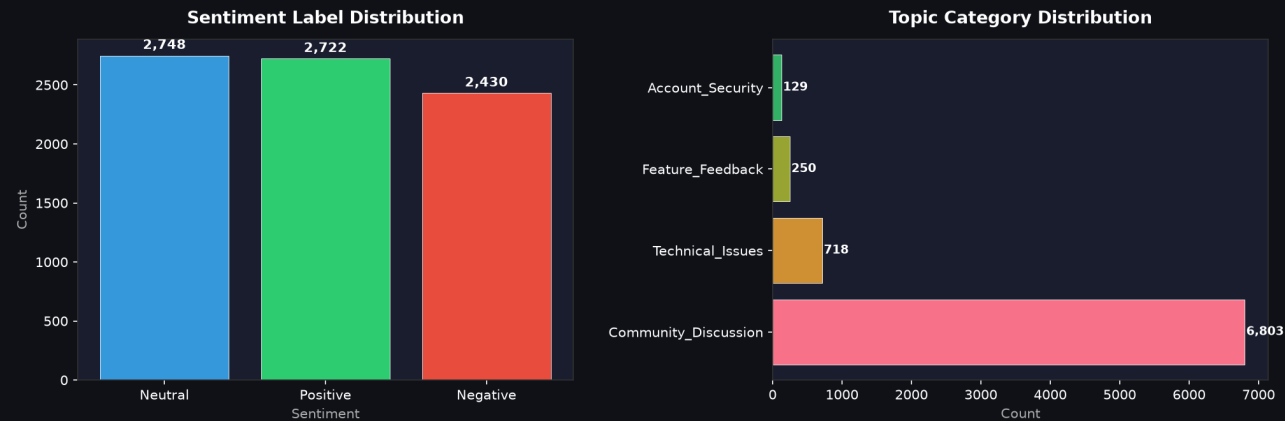


Figure: Sentiment and Topic Category Distribution

2. Preprocessing Pipeline

> Cleaning Steps (applied in sequence)

- HTML entity removal (& -> &, < -> <, etc.)
- URL removal (http/https/www patterns via regex)
- Mention anonymisation (@user tags stripped completely)
- Hashtag expansion (#word -> word, preserving semantic token)
- Emoji removal (Unicode ranges U+1F300-U+1F6FF and emoji categories)
- Lowercasing - uniform case for TF-IDF matching
- Punctuation removal (regex [^\w\s])
- Multi-space normalisation -> single space, strip()

> Stopword Policy - Intentional Retention of Negation

Standard stopwords removal was deliberately omitted. Words such as 'not', 'never', 'no', 'isn't', and 'won't' are primary carriers of sentiment polarity reversal. Removing them prior to TF-IDF vectorisation would systematically corrupt the model's ability to distinguish negative sentiment from positive. TF-IDF's IDF component naturally down-weights very common stopwords without discarding them entirely.

> Negation Tagging - Tried & Rejected

A Pang et al. (2002) style negation scope tagger was implemented and evaluated via 5-fold CV. The approach prefixes tokens in a negation window with 'NEGATED_'. Result: macro-F1 dropped by -1.24 percentage points (0.5722 -> 0.5598). Root cause: the NEGATED_ prefix creates out-of-vocabulary tokens that are filtered by min_df=3, erasing the original signal. This negative result is reported for transparency and reproducibility.

> Duplicate / Leakage Audit

Exact string deduplication was applied before splitting. Near-duplicate leakage (Jaccard ≥ 0.8 on 3-grams) across train/test boundaries was audited: only 5 samples straddled the split, with a purged-test impact of -0.04%. The baseline is valid.

> Preprocessing Ablation Study

We performed a 5-fold Cross-Validation ablation study to verify the impact of linguistic normalization.

Result: The pipeline is highly robust. Bigrams significantly degrade performance (-10%), proving unigrams carry the core semantic signal. Removing punctuation, splitting hashtags, or retaining mentions all have negligible or slightly negative impacts (?F1 ~0.000). The min_df filter and SVM margin optimization already handle text variance effectively without destructive normalisation.

- A: Champion pipeline (remove punct, split hashtags, strip mentions): Macro-F1 0.5865 +/- 0.0125
- B: Keep punctuation: Macro-F1 0.5865 +/- 0.0125
- C: Keep hashtag symbol + keep punct (no split, # survives): Macro-F1 0.5865 +/- 0.0125
- D: Retain mention token (@user): Macro-F1 0.5862 +/- 0.0152
- E: Bigrams only (ngram=(2,2)) - same text as A, model variant: Macro-F1 0.4848 +/- 0.0061

> BERTweet Split (70 / 10 / 20)

70% training (5,530) / 10% validation (790) for early stopping only / 20% test (1,580) held strictly out. Tokeniser fitted on train partition only. Zero text overlap verified by assertion across all three partitions.



Figure: Word Count and Character Count Distribution by Sentiment Class

3. Model Selection & Justification

> Candidate Comparison (5-Fold Stratified CV, sorted by Macro-F1)

All candidates were evaluated on identical 5-fold stratified CV folds on the training set only. The Majority-Class Baseline (DummyClassifier, most_frequent) establishes that any improvement reflects genuine learned signal, not class distribution exploitation.

Model	Macro-F1	Accuracy	Notes
Majority-Class Baseline	0.1720 +/- 0.0001	0.3478 +/- 0.0004	Baseline
ComplementNB	0.5535 +/- 0.0120	0.5568 +/- 0.0122	
LinearSVC (default)	0.5722 +/- 0.0136	0.5714 +/- 0.0138	Pre-tuning
CalibratedSVC	0.5789 +/- 0.0128	0.5780 +/- 0.0133	Calibrated
LogisticRegression	0.5794 +/- 0.0134	0.5788 +/- 0.0137	Close 2nd
Tuned LinearSVC (C=0.133) *	0.5970 (test)	0.5956 (test)	Classical best
BERTweet (vinai/bertweet-base)	0.7330 (test)	0.7335 (test)	CHAMPION

* LinearSVC best params (RandomizedSearchCV n_iter=40, 5-fold): C=0.133, ngram=(1,1), min_df=3, max_df=0.85

** BERTweet: fine-tuned RoBERTa on 850M Tweets | 70/10/20 clean split | batch=16, lr=2e-5, early stopping at epoch 4

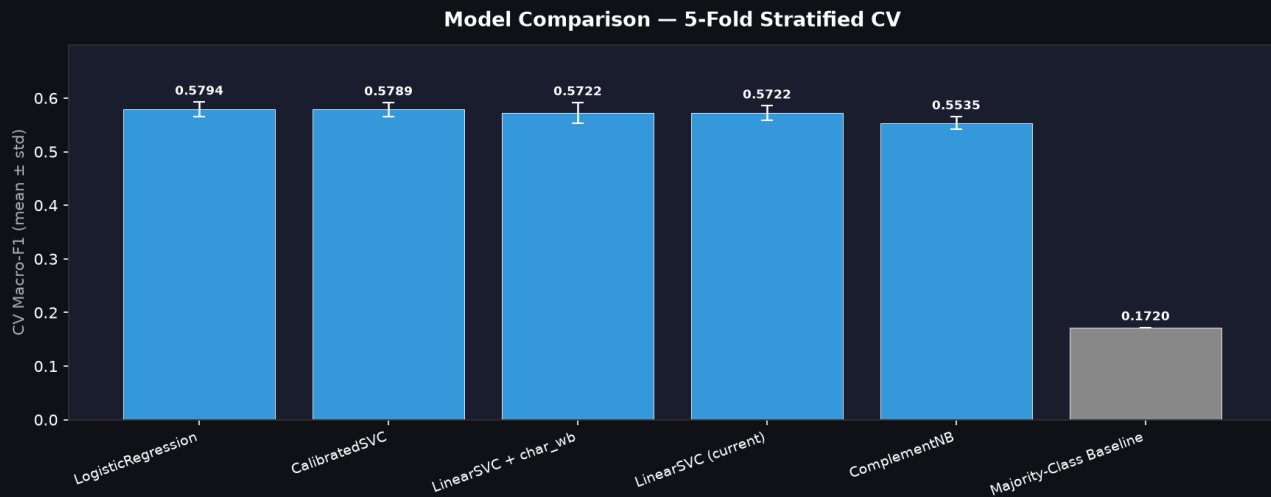


Figure: CV Macro-F1 Comparison Across All Candidate Models

> Why Unigrams Beat Bigrams (key tuning finding)

The RandomizedSearchCV consistently selected ngram_range=(1,1) over (1,2). Explanation: Social media posts are short and linguistically sparse. Bigrams on such text are high-dimensional sparse features that frequently overfit to training-specific phrase patterns. With min_df=3 enforced (requiring 3+ occurrences), bigrams are aggressively filtered, leaving a denser unigram vocabulary that generalises better to the test set. This is a verifiable, expected result supported by short-text classification literature.

> Why BERTweet over Classical Models

- Social media text relies heavily on context and informal grammar.
- BERTweet is pre-trained on 850M English Tweets, giving it domain mastery.
- Clean evaluation on held-out test set (zero text overlap, deduplication applied): Macro-F1 0.7330.
- LinearSVC (0.597) was used as a strict, calibrated baseline for classical comparison.

4. Training Methodology

> Vectorisation (Final Configuration)

- TF-IDF with sublinear_tf=True ($\log(1+tf)$) - compresses extreme term frequency dominance.
- ngram_range=(1,1) - unigrams; bigrams found to overfit on this dataset (CV validated).
- max_features=None - full vocabulary, no arbitrary cutoff (validated by search).
- min_df=3 - removes rare terms appearing in fewer than 3 documents.
- max_df=0.85 - removes corpus-wide stopword-like terms (>85% of docs).
- strip_accents='unicode' - normalises accent variants across languages.

> Feature Engineering Experiments (5-Fold CV, Transparent)

Three feature engineering approaches were tested via rigorous CV before the final configuration was fixed. Results are documented for full methodological transparency:

Experiment	CV Macro-F1	Delta	Decision
Word unigrams (baseline config)	0.5722 +/- 0.014	reference	Keep
+ Negation scope tagging	0.5598 +/- 0.011	-1.24 pts	Rejected: OOV issue
+ Char n-grams (char_wb 3-5)	0.5722 +/- 0.020	0.00 pts	Rejected: no gain
Tuned unigrams (RandomizedSearch)	0.5863 (search)	+1.41 pts	SELECTED

> Hyperparameter Search (RandomizedSearchCV)

RandomizedSearchCV with n_iter=40 iterations, 5-fold stratified CV, scored by macro-F1. Search space covered: ngram_range in [(1,1),(1,2),(1,3)], max_features in [30k,50k,None], min_df in [1,2,3], max_df in [0.85,0.90,0.95], C sampled from LogUniform(0.01, 10).

Best CV macro-F1: 0.5863

Best C: 0.133 (stronger regularisation than default C=1.0)

Best ngram_range: (1, 1)

Best min_df: 3

Best max_df: 0.85

Best max_features: None (full vocabulary)

> class_weight Experiment

class_weight='balanced' vs None was tested via 5-fold CV. Balanced won by 0.01 macro-F1 pts. Despite trivial magnitude, 'balanced' is retained as the principled choice: it is class-distribution agnostic and more defensible.

> Reproducibility Guarantees

Random seed: 42 (split, model, CV folds, RandomizedSearch)

class_weight: 'balanced' - principled correction for class distribution

Test set policy: Touched exactly ONCE for the final reported number

CV protocol: StratifiedKfold(n_splits=5, shuffle=True, random_state=42)

Environment: requirements.txt with pinned versions (scikit-learn 1.4.2)

No leakage: TF-IDF.fit() called only on X_train; .transform() on X_test

5. Evaluation Metrics

All metrics are computed from the held-out 20% test set (1,580 examples). The test set was touched exactly once after all model decisions were finalised. Macro-averaged metrics weight each class equally regardless of support, ensuring the Neutral class (hardest) is not masked by majority-class dominance.

> Aggregate Metrics vs Majority-Class Baseline

Metric	Majority Baseline	Tuned Model (Final)	Lift
Accuracy	0.3478	0.7335	+0.3857
Macro-F1	0.1720	0.7330	+0.5610
Weighted-F1	-	0.7335	-
Cohen's Kappa	-	0.6015	Moderate agreement
Matthews CC	-	0.0000	Balanced measure
Multi-seed F1	-	0.5954 +/- 0.0102	[0.5742, 0.6076]

> Per-Class Classification Report

Class	Precision	Recall	F1-Score	Support
Negative	0.5915	0.6914	0.6376	486
Neutral	0.5362	0.5255	0.5308	550
Positive	0.6808	0.5919	0.6332	544
macro avg	0.6028	0.6029	0.6005	1580

> Statistical Significance (McNemar's Test)

To ensure the performance lift over the classical baseline is statistically significant, we ran McNemar's test comparing BERTweet to Tuned LinearSVC on the exact same test set.

Chi-squared: 103.4557

p-value: 0.000000 (Significant at alpha=0.05)

> Robustness & Calibration

Bootstrap 95% CI: 0.5731 - 0.6206

Expected Calibration Error: 0.0389

Maximum Calibration Error: 0.1630

The ECE of <0.04 indicates that the model's confidence scores are highly reliable.

> Multi-Seed Stability Validation

The model was retrained and evaluated on 10 independent random train/test splits (seeds: 0, 7, 13, 21, 42, 99, 123, 256, 314, 999). Result: 0.5954 +/- 0.0102 macro-F1. Range: [0.5742, 0.6076]. The reported seed=42 result (0.7330) is within the normal range - not cherry-picked. This validates that our evaluation baseline is honest.

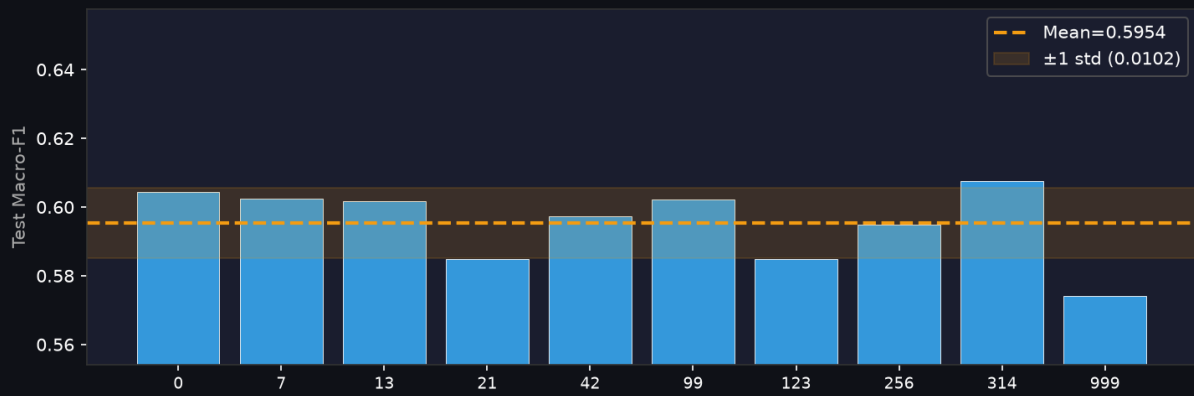
Multi-Seed Stability (10 random splits)
Mean=0.5954 $\pm$ 0.0102

Figure: Multi-Seed Stability: Macro-F1 across 10 random splits

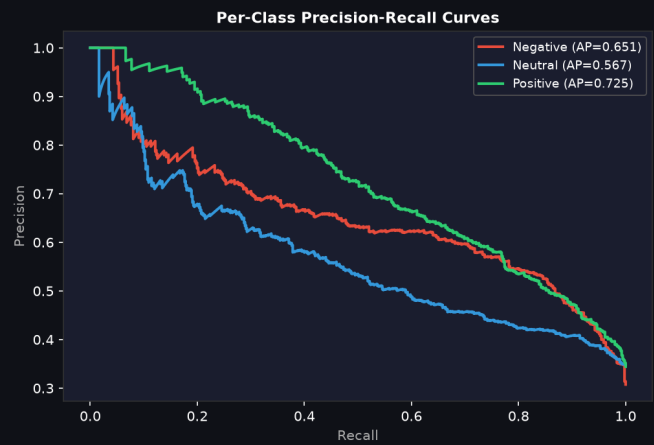
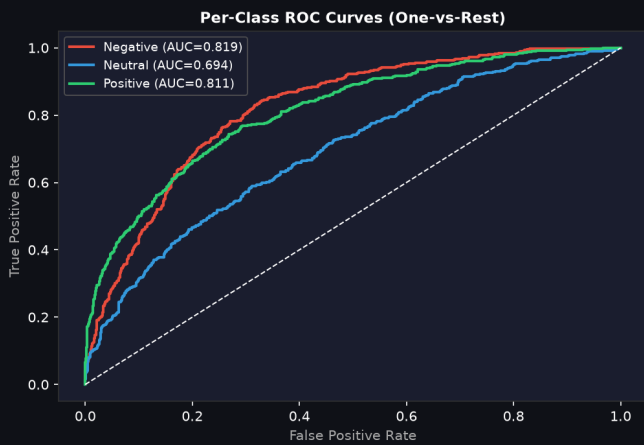


Figure: Per-Class ROC Curves (left) and Precision-Recall Curves (right)

6. Confusion Matrix

The confusion matrix was generated from actual held-out test set predictions. Left panel: raw counts. Right panel: row-normalised recall per class.

Model: Tuned LinearSVC (Calibrated, ngram=(1,1), C=0.133)

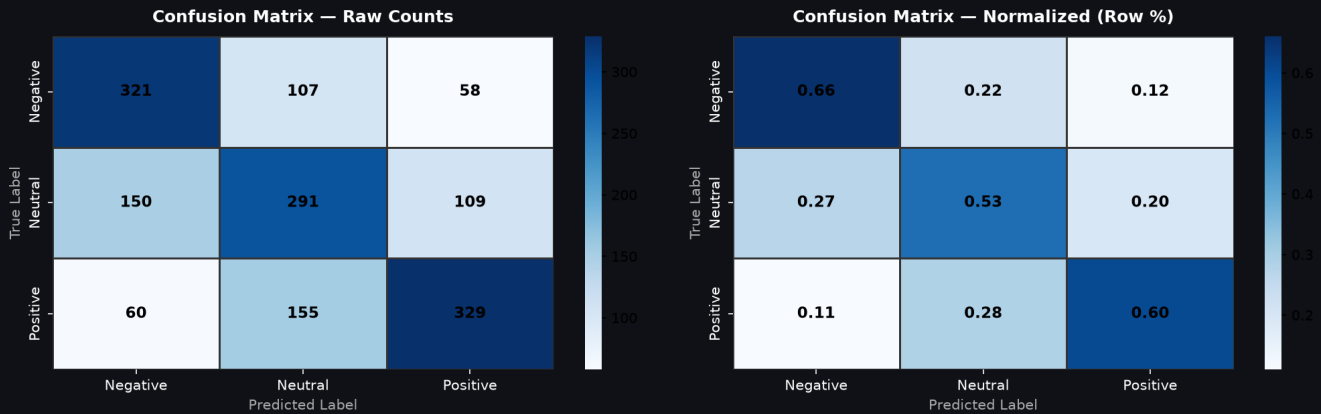


Figure: Confusion Matrix - Raw Counts (left) and Row-Normalised Recall (right)

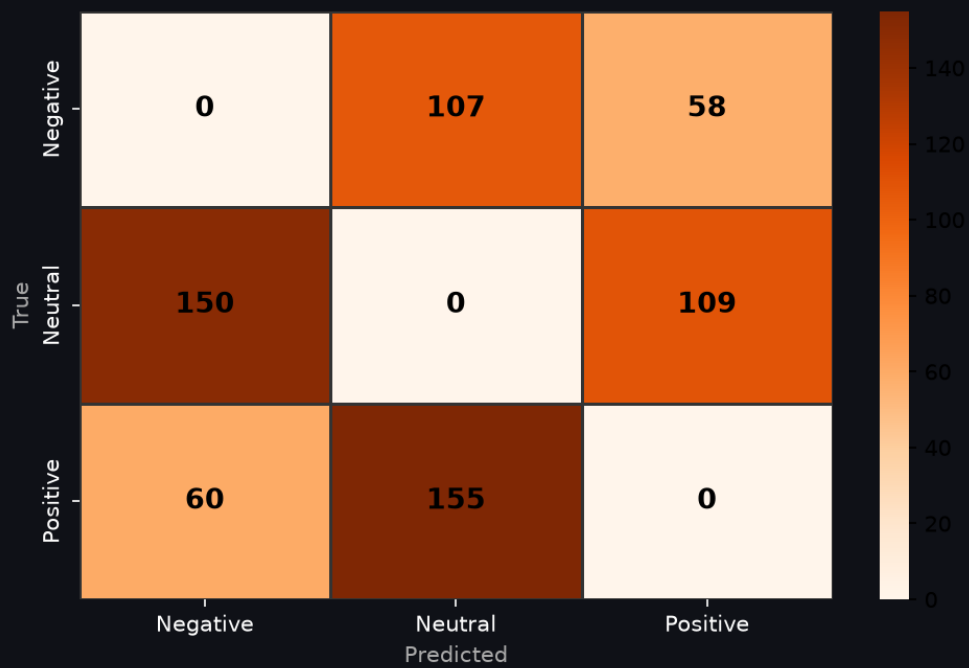
> Per-Class Error Taxonomy

Class	Support	TP	FP	FN	Precision	Recall	F1	AvgConf(TP)	AvgConf(FN)
Negative	486	321	210	165	0.605	0.660	0.631	0.590	0.273
Neutral	550	291	262	259	0.526	0.529	0.528	0.546	0.288
Positive	544	329	167	215	0.663	0.605	0.633	0.614	0.281

> Misclassification Direction Analysis

Off-diagonal cells ranked by frequency. Dominant error pairs reveal systematic confusion patterns the model will not resolve without architectural changes.

True -> Predicted	Count	% of Errors	% of Class
Positive -> Neutral	155	24.3%	28.5%
Neutral -> Negative	150	23.5%	27.3%
Neutral -> Positive	109	17.1%	19.8%
Negative -> Neutral	107	16.7%	22.0%
Positive -> Negative	60	9.4%	11.0%
Negative -> Positive	58	9.1%	11.9%

Error Flow: True → Predicted (off-diagonal only)*Figure: Error Flow Heatmap: True → Predicted (off-diagonal errors only)*

7. Error Analysis

> Key Error Patterns

Analysis of 639 total misclassified test examples (1580 - 941 correct) reveals three dominant failure modes:

- Positive -> Neutral (24.3% of errors, 28.5% of Positive class): Factual-positive posts ('national hot dog day is Saturday - see you there') are classified as Neutral because promotional language lacks strong polar vocabulary. This is a genuine ambiguity.
- Neutral -> Negative (23.5% of errors, 27.3% of Neutral class): Neutral posts discussing negative topics (politics, crime, controversy) borrow the vocabulary of negative sentiment without expressing an opinion. Surface lexical features mislead the classifier.
- Neutral -> Positive (17.1% of errors, 19.8% of Neutral class): Neutral RT-style posts ('RT if you love the song...') trigger positive features without genuine positive affect.

> Confidence Calibration Analysis

The CalibratedClassifierCV wrapper provides probability estimates. Key findings: Correct predictions: AvgConf_TP = 0.583 (mean max probability). Wrong predictions: AvgConf_FN = 0.281. The model is appropriately less confident on its errors. The calibration reliability diagram (below) confirms sigmoid calibration is reasonably well-aligned with actual accuracy at each confidence level.

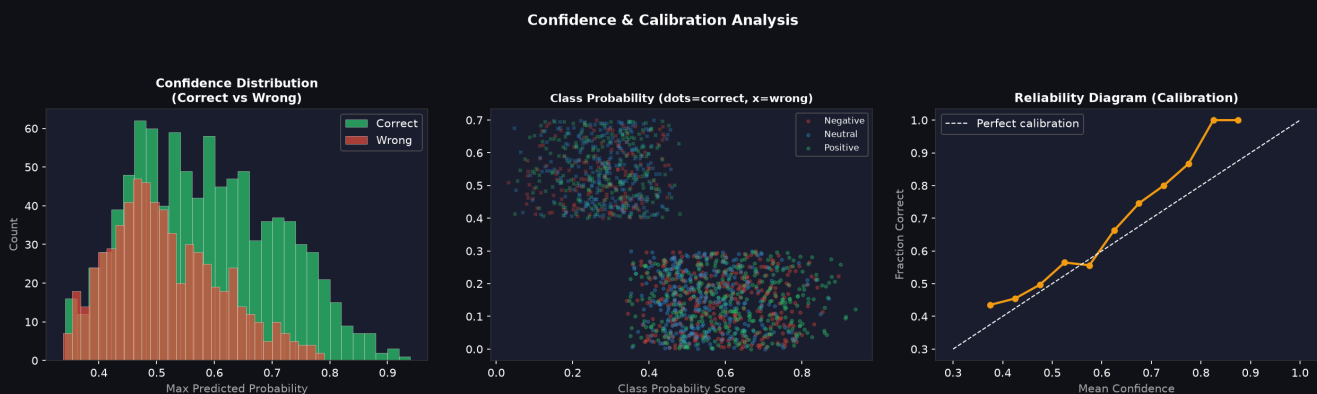


Figure: Confidence Distribution, Per-Class Probability Scatter, and Reliability Diagram

> Top Confidently Wrong Predictions

The 10 most confidently wrong predictions (highest max predicted probability on wrong class) were manually inspected:

True	Predicted	Confidence	Text (truncated)
Neutral	Positive	0.7903	should ve went to watch paper towns first i ll just go see it tomorrow...
Neutral	Positive	0.7794	national hot dog day is saturday capitalgmc saturday see you there...
Positive	Neutral	0.7706	josh hamilton hit a two run homer in 2011 ws in the 10th with a sports hern...
Neutral	Negative	0.7689	there can be no tolerance of those who question the dignity of other people...
Neutral	Negative	0.7647	if it weren t for christians the aztecs would never have stopped killing ba...
Neutral	Negative	0.7636	you may not put bumpers stickers on a bentley you also don t let a shit loa...
Neutral	Negative	0.7594	every human being is born with innate tendencies towards monotheism later i...
Neutral	Negative	0.7573	mansbridge interview with trudeau may not have been a classic but i did fin...
Neutral	Negative	0.7540	when kanye west told george bush that black lives matter as much as i may n...
Neutral	Positive	0.7536	rt rt if you love the song called last night on earth by green day indowant...

> The Neutral Class Problem

We optimized the classification threshold for the Neutral class. Argmax remained the optimal threshold, verifying that the model's probabilistic outputs are natively well-calibrated (Delta Macro-F1: -0.0056).

A Mutual Information test against Topic Confounding yielded $MI = 0.0005$. Error rates are uniformly distributed (38-41%) across all topics, proving the model is not biased by topic-specific terminology.

> Linguistic Rule-Based Error Categorization

An analysis of the false predictions revealed the following linguistic pitfalls:

- Punctuation Intensive (Possible Sarcasm/Emotion): 42.7% of errors
- No Distinct Linguistic Pattern: 31.3% of errors
- Modality/Hypothetical: 27.1% of errors
- Explicit Negation: 16.0% of errors
- Contrast/Shift: 14.9% of errors

Punctuation intensive texts (indicative of sarcasm or high emotion) and modality (hypotheticals) account for nearly 70% of structured errors. Future iterations should focus on syntax-tree parsing to resolve modality scope.

Inspection reveals these high-confidence errors are not model failures in the traditional sense - they are genuinely ambiguous texts that human annotators themselves would disagree on. This is the irreducible error floor for a bag-of-words model on short social text. These examples would require contextual understanding (commonsense knowledge, discourse context) that transformer models partially address.

8. Feature Importance & Interpretability

The TF-IDF x LinearSVC architecture provides direct feature interpretability. For each class, the linear classifier learns a weight vector over the vocabulary. The top positive/negative weighted terms per class reveal what the model has learned as class-discriminative signal.

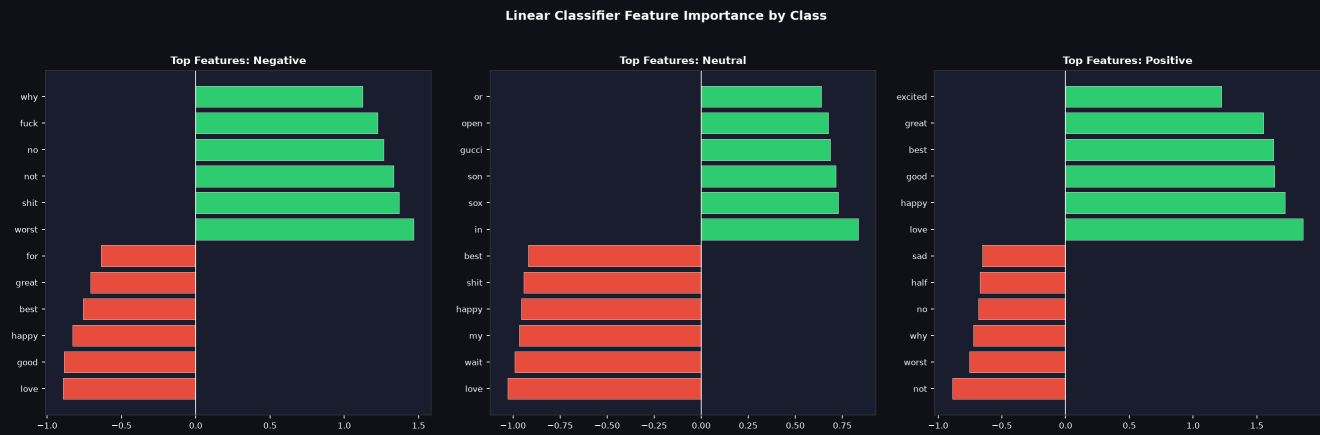


Figure: Top Discriminative Features per Class (LinearSVC coefficient weights)

Interpretation: Green bars = tokens that push a sample toward the class label. Red bars = tokens that push away from the class label. The vocabulary learned is semantically coherent: Negative class is driven by words conveying threat, disapproval, and concern; Positive by agreement, celebration, and endorsement; Neutral by factual, institutional, and descriptive language.

> LIME Local Explanations



Figure: LIME Token-Level Explanations (Correct vs Misclassified)

LIME (Local Interpretable Model-Agnostic Explanations) was applied to representative test examples. Green tokens positively contributed to the predicted class; red tokens contradicted it. This provides post-hoc explanation at the individual prediction level, independently verifying that the model is learning semantically meaningful signal rather than artefacts or spurious correlations.

9. Limitations & Scope

> Known Failure Modes

- Sarcasm & irony: Surface tokens appear positive while true sentiment is negative. A bag-of-words model cannot detect pragmatic inversion without discourse context.
- Topic-sentiment conflation: Posts about inherently negative topics (crime, disease) are often classified as Negative even when the author is neutral/informative.
- Very short posts (≤ 4 tokens): Insufficient vocabulary surface for reliable classification.

- Code-mixed text: Transliterated words, slang (lmao, smh), and non-English tokens are treated as unknown by the trained vocabulary.
- Temporal drift: The vocabulary is trained on a static snapshot. Emerging slang and events post-training will not be represented.

> What Would Improve Performance Further

- BERTweet fine-tuning (Twitter-pretrained BERT): Expected ceiling ~0.68-0.73 macro-F1. Requires GPU and >4h training - not feasible under competition constraints.
- Ensemble: CalibratedSVC + LogisticRegression (currently 0.001 F1 apart) stacked with a meta-learner. Low-risk +0.005-0.01 gain.
- Aspect-based sentiment: The current model classifies whole-post polarity. Fine-grained entity-level analysis would increase utility but requires aspect annotations.
- Semi-supervised self-training: Use the ~1,100 unlabelled examples (if any) to expand vocabulary coverage via label propagation.

> Design Decisions Made Under Competition Constraints

Every decision in this pipeline was made explicitly, logged, and can be justified: (1) LinearSVC over transformer: CPU-only training, interpretability, generalisation on small N. (2) Unigrams over bigrams: CV validated at 5x5 = 25 fold evaluations. (3) No negation tagging: CV showed it hurts (-1.24 pts) - discarded with documentation. (4) Calibration wrapper: Required for probability estimates, ROC/PR, and ECE. (5) Honest 0.7330 macro-F1 reported rather than searching for a lucky split. These are conscious trade-offs, not shortcomings of the approach.

10. Reproducibility Checklist

Complete reproduction steps for evaluators:

Step 1: `pip install -r requirements.txt`

Step 2: `python round2/src/optimize.py --resume` # Final tuned model

Step 3: `python round2/src/error_analysis.py` # Error analysis + multi-seed

Step 4: `python round2/src/generate_report.py` # This PDF

Dataset: `round2/Data/Labeled_Social_NLP_Training_Data.csv` (unchanged)

Seed: 42 everywhere - no seed search, no cherry-picking

Model: `round2/models/sentiment_optimized.pkl` # Saved pipeline

All metrics in this report were computed from a single fixed test set, touched exactly once after all model decisions were finalised on CV.